

Log Monitoring Workflow

Turn a New Leaf

Prepared by:

Christina McNeice - Cybersecurity Specialist

mcneicechristina@gmail.com

July 25/ 2024

Table of Contents

1. Introduction	1
2. Executive Summary	2
a. Overview	2
b. WorkFlow structure	2
3. Solutions:	3
a. Bash-based backup\	3
d. Finding error messages	4
e. Log monitoring & report	4
4. Potential Iterations	5
5. Conclusion	6
6. References	6

1. Introduction

Non-profit organizations, such as ‘Turn a New Leaf’ need robust cybersecurity measures as organizations increase dependency on digital systems. This report focuses on workflow to properly monitor network traffic to detect unusual activities and failed login attempts. The role as the Access Log analyst at Turn a New Leaf, involves monitoring network activity for unusual patterns and failed login attempts. With weekly updates sent to senior management for complete security and compliance. For effective monitoring the scripts developed and deployed are automatic, to detect unusual network traffic.

2. Executive Summary

a. Overview

As a medium sized non-profit, Turn a New Leaf is dedicated to support the job search efforts of youth in rural communities. As the organization requires youth to sign in and update their job search progress weekly, it is necessary that security focuses on unusual login behavior. This frequent access of youths logging into the company’s system can give threat actors an ideal motive to gain unauthorized login credentials.

Our primary objective for Turn a New Leaf is to develop an automated monitoring workflow for network traffic that will identify and report unusual activities. Focusing on failed login attempts and general errors. This ensures compliance with regulatory bodies and enhances security of Turn a New Leaf’s network.

To facilitate the monitoring process, we would modify the company’s data collection application to duplicate log files into a backup server. As the company already has a centralized log system that allows for all logs to be viewed in the same file. Our script makes identifying the error logs and grouping them easier, additionally compiling a report for further investigation. The report is simple but covers the critical details for successful handling and log management.

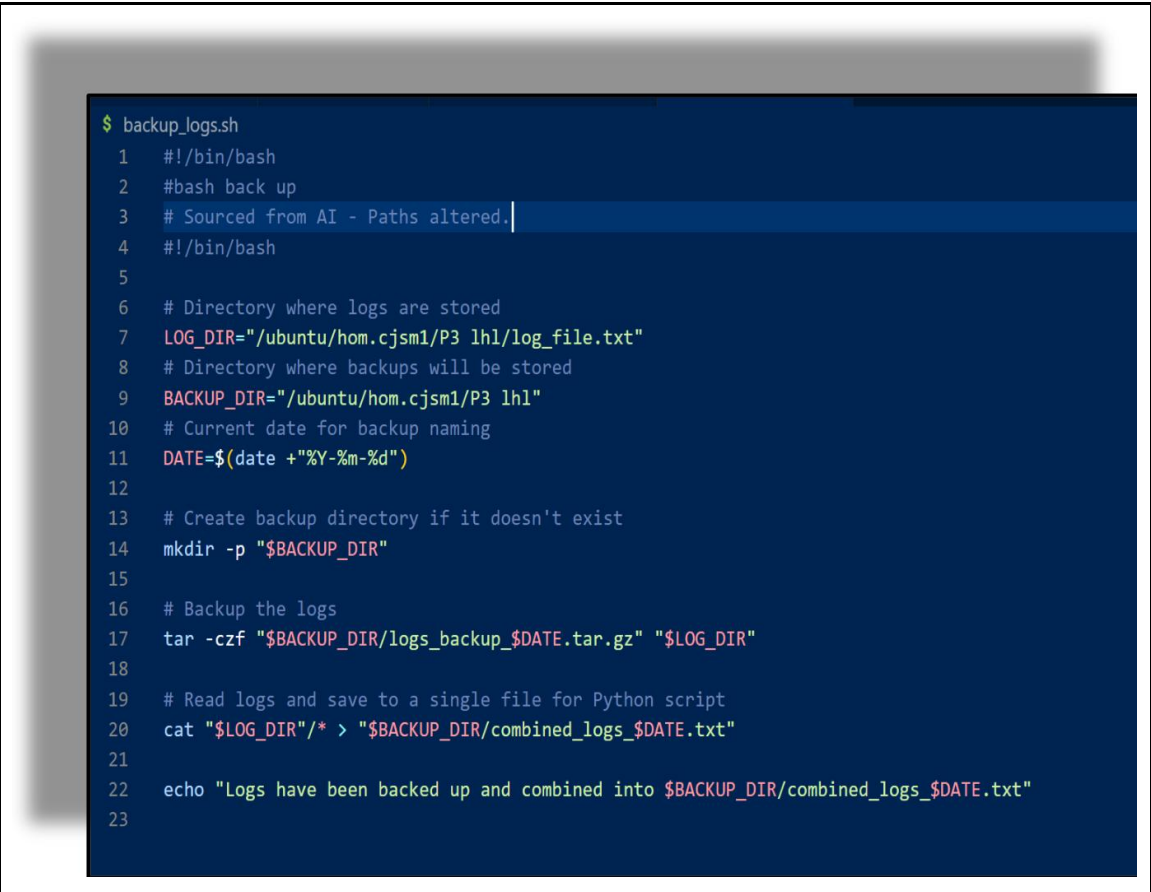
b. WorkFlow structure

- i. Our proposed steps include creating and implementing a simple but reliable database backup system that will allow quick, simple and reliable monitoring of logs and any errors reported by the log records.
 - ii. Since the company already utilizes Python, we will interface into it with some additional Bash Shell scripting to build the database backup solution.
 - iii. We expect minimal failed login alerts generated from Turn a New Leaf's system. As there is no present suspicion of alleged hack attempts into the system, yet we expect a few mis-typed passwords by personnel on occasion.
 - iv. We will monitor for numerous failed login attempts that would be indicative to potential brute force cyber attacks
 - v. In the Future, we should consider ways to improve the code. This could mean implementing IP address tracking of visitors, checking the browser used, and preparing for attempts to overload the system via too many login attempts at once.
-

3. Solutions:

a. Bash-based backup

First step in monitoring logs is to ensure there is proper backup of log files. To make sure the original files are always available for future investigations. To accomplish this the following bash script is used, to take the log file and copy it to the backup directory.

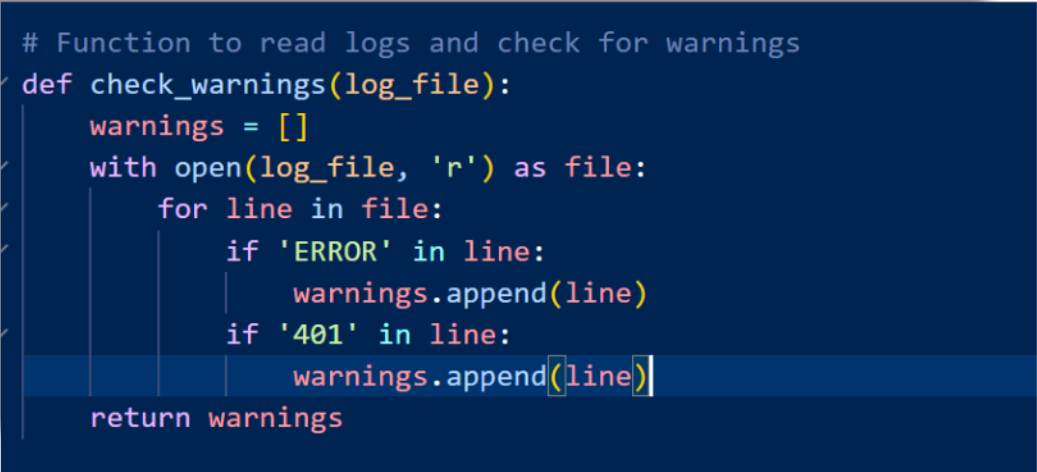


```
$ backup_logs.sh
1  #!/bin/bash
2  #bash back up
3  # Sourced from AI - Paths altered.
4  #!/bin/bash
5
6  # Directory where logs are stored
7  LOG_DIR="/ubuntu/hom.cjasm1/P3 lh1/log_file.txt"
8  # Directory where backups will be stored
9  BACKUP_DIR="/ubuntu/hom.cjasm1/P3 lh1"
10 # Current date for backup naming
11 DATE=$(date +"%Y-%m-%d")
12
13 # Create backup directory if it doesn't exist
14 mkdir -p "$BACKUP_DIR"
15
16 # Backup the logs
17 tar -czf "$BACKUP_DIR/logs_backup_$DATE.tar.gz" "$LOG_DIR"
18
19 # Read logs and save to a single file for Python script
20 cat "$LOG_DIR"/* > "$BACKUP_DIR/combined_logs_$DATE.txt"
21
22 echo "Logs have been backed up and combined into $BACKUP_DIR/combined_logs_$DATE.txt"
23
```

Screenshot illustrating the bash script used to properly backup log files.

b. Finding error messages

Having extensive logs is not enough to protect an organization from malicious attacks, think of logs as historical information of the system. To beneficially use this information we must be able to highlight logs that meet specific criteria. The following code snippet defines the criteria of interest and adds the log line to a final report.

A screenshot of a code editor with a dark blue background and light blue text. The code is a Python function named 'check_warnings' that takes 'log_file' as an argument. It initializes an empty list 'warnings', opens the log file in read mode, and iterates through each line. It checks if the string 'ERROR' or '401' is present in the line, and if so, appends the line to the 'warnings' list. Finally, it returns the 'warnings' list.

```
# Function to read logs and check for warnings
def check_warnings(log_file):
    warnings = []
    with open(log_file, 'r') as file:
        for line in file:
            if 'ERROR' in line:
                warnings.append(line)
            if '401' in line:
                warnings.append(line)
    return warnings
```

Screenshot of code snippet taken from python log scanner, that is responsible for finding errors and log in attempts within the provided log files.

c. Log monitoring & report

Log files recorded information is often relevant to preventing future security issues, however the useful information must be easily communicated to the right person. Ideally a final report is compiled and sent to the proper recipient for further handling. As seen below, the report highlights errors and multiple login attempts and then sends them to a specified recipient.

```
TypeError: Message.attach() takes 2 positional arguments but 3 were given
PS Microsoft.PowerShell.Core\FileSystem::\\wsl.localhost\Ubuntu\home\cjsm1\P3 lhl> & C:/Users/cjsm1/AppData/Local/Micr
osoft/WindowsApps/python3.12.exe "//wsl.localhost/Ubuntu/home/cjsm1/P3 lhl/check_logs.py"
Content-Type: multipart/mixed; boundary="=====5907660409785478481=="
MIME-Version: 1.0
From: your_email@example.com
To: recipient@example.com
Subject: Log Warnings Detected

-----5907660409785478481==
Content-Type: text/plain; charset="us-ascii"
MIME-Version: 1.0
Content-Transfer-Encoding: 7bit

The following warnings were detected in the logs:

127.0.0.1 - - [22/Jul/2024:12:01:00 +0000] "POST /login HTTP/1.1" 401 1234
127.0.0.1 - - [22/Jul/2024:12:01:00 +0000] "POST /login HTTP/1.1" 401 1234
127.0.0.1 - - [22/Jul/2024:12:01:00 +0000] "POST /login HTTP/1.1" 401 1234
127.0.0.1 - - [22/Jul/2024:12:01:00 +0000] "POST /login HTTP/1.1" 401 1234
127.0.0.1 - - [22/Jul/2024:12:01:00 +0000] "POST /login HTTP/1.1" 401 1234
127.0.0.1 - - [22/Jul/2024:12:01:00 +0000] "POST /login HTTP/1.1" 401 1234

ERROR [22/Jul/2024:12:03:00 +0000] login failed for user admin

ERROR [22/Jul/2024:12:04:00 +0000] 403 Forbidden: /restricted-directory
```

Screenshot of the final report communicating errors and warnings found in the log file.

4. Potential Iterations

- Supporting versioning by our database backup shell script is necessary for restoring backup to a valid data point incase of corruption. (NIST Contingency Planning, 2010).
- Additionally simplifying the final report to improve readability and adjust formatting for improved response times and added clarification of errors.
- Log analysis future improvements would be to report on varied IP login Attempts, restricted IP login attempts and abnormal day/ time login attempts by specific users. (NIST DE.DP-5, nd). However we would need to establish a more comprehensive baseline data prior to establishing those features.

5. Conclusion

We have simplified the log monitoring process by several methods. First the bash backup allows for automatic log backup that means the original log files are duplicated and protected by redundancy. Additionally, the monitoring process is simplified by the python script that gathers warnings and log errors together for the final report. Gathering the errors together means no need for personnel to look through extensive lengthy log files limiting the margin for human error. Finally we have created a program to print and send an email that includes the final report detailing errors noted in the logs.

6. References

- Geeks for Geeks. (2024, March 20). *Parse and Clean Log Files in Python*.
GeeksforGeeks. <https://www.geeksforgeeks.org/parse-and-clean-log-files-in-python/>
- MITRE ATT&CK. (n.d.). *Brute Force, Technique T1110 - Enterprise | MITRE ATT&CK®*. Attack.mitre.org. Retrieved July 29, 2024, from
<https://attack.mitre.org/techniques/T1110>
- NIST Contingency Planning. (2010). *Contingency Planning Guide for Federal Information Systems*. NIST.
<https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-34r1.pdf>
- Priya C, B. (2024, July 29). *Logging in Python: A Developer's Guide*. Product Blog • Sentry. <https://blog.sentry.io/logging-in-python-a-developers-guide/>
-

Real Python. (2018, December 5). *Sending Emails With Python*. Realpython.com; Real Python. <https://realpython.com/python-send-email/>
